

Harness Engineering 驾驭工程

AI Agent 时代的
软件工程新范式

同一模型更换架构后成功率

~~42%~~ → **78%**
常规 Prompt Harness 赋能

控制论闭环示意



基于顶级AI 团队对 harness engineering 的认识的综合分析

"Zero-Handwritten Code"

OpenAI Codex 团队 Ryan Lopopolo, 实现百万行代码零手写。

"The AI Adoption Journey"

Mitchell Hashimoto (Terraform 之父), AI 采纳实战总结。

"Long-Horizon Task Harness"

Anthropic Claude 4.5 团队, 长任务 Harness 架构的奠基人与实践者。

"Harness is Cybernetics"

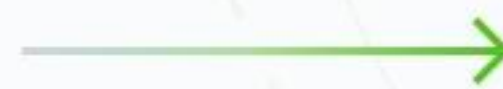
GeorgeZhang, 控制论专家, 首次提出 Harness 的本质是控制论。

01

为什么 Harness Engineering 突然爆红？

“不是媒体炒作，
是生产环境逼出来的。”

手动调节



系统控制

真实业务的毒打：Demo 到生产的百倍落差

 短跑 Demo
理想沙盒

VS

 马拉松级生产环境
残酷现实

任务形态

长程任务，跨多次会话与多天迭代

代码场域

万行级历史资产，直面异质技术债

交付标准

绝对稳定性，死守系统安全边界

容错成本

极易产生不可逆 AI 垃圾与架构破坏

核心诉求

追求复杂系统下的艰难「收敛」

交互极短，单次调用即可见效

干净的微型项目，约束条件极弱

跑通即胜利，能录屏演示即可

试错成本趋零，大不了清空重跑

追求单点极致的「生成」

“ 环境的真实复杂度，将模型能力的微小波动残忍放大了 **100** 倍。 ”

Agent 生产力的「崩坏四部曲」

典型痛感清单：从初见惊艳到系统腐烂

🔪 初见惊艳

Demo 跑通，功能勉强可用，掩盖了边界缺失。

🔑 风格变形

同一问题出现多种实现，架构逐渐“糊”在一起。

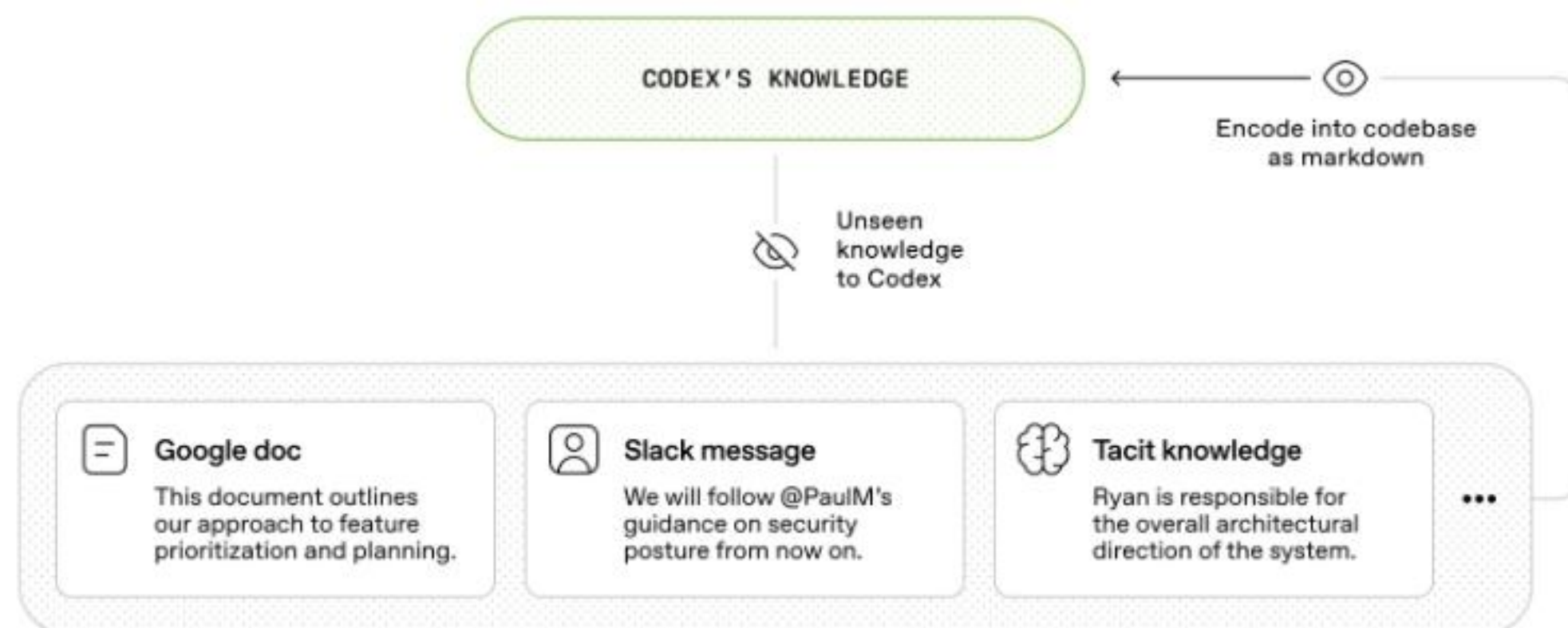
🔄 循环跌倒

相同 bug 反复出现，错误经验无法系统化。

💩 系统腐烂

堆满半成品、冲突文档，产生大量“AI 垃圾”。

The limits of agent knowledge:
What Codex can't see doesn't exist



🖼️ Agent 知识的局限性与隐性人类知识鸿沟

⚠️ 最终后果

工程师从「系统设计者」
退化为「**AI 垃圾分拣员**」

🔍 崩坏根因

缺乏对 Agent 行为的 **持续约束与反馈**。当未见信息无法被有效编码入库时，系统必然走向失控。

AI Slop: 规模化生产的“系统熵增”

Agent 7x24 小时狂奔下的代码废料危机

生产模式降维对比

评估维度	人类工程师	 AI Agent
生成速度	慢 (受限于人力)	极快 (7x24 小时)
错误扩散	局部、易被察觉	全局 (分钟级全库复制)
维护成本	线性增长	指数级增长 (剧烈熵增)

AI Slop 临床表现

-  **过时文档**
与真实代码行为完全脱节的说明，误导后续开发
-  **坏模式扩散**
一个错误的 API 调用姿势，瞬间被复制到全仓库
-  **表面完成**
代码写满屏幕，但底层逻辑根本无法过上线标准

“

没有 Harness 的系统，工程定律
熵增速度等于 AI 的生成速度。

”

关键矛盾：难点不在“生成”，而在“校准、验证、约束、清理”

— 从“生成工程”转向“控制工程”

🔧 校准

持续评估偏差，建立纠偏机制，避免 Agent 在复杂任务中随机起飞。

✅ 验证

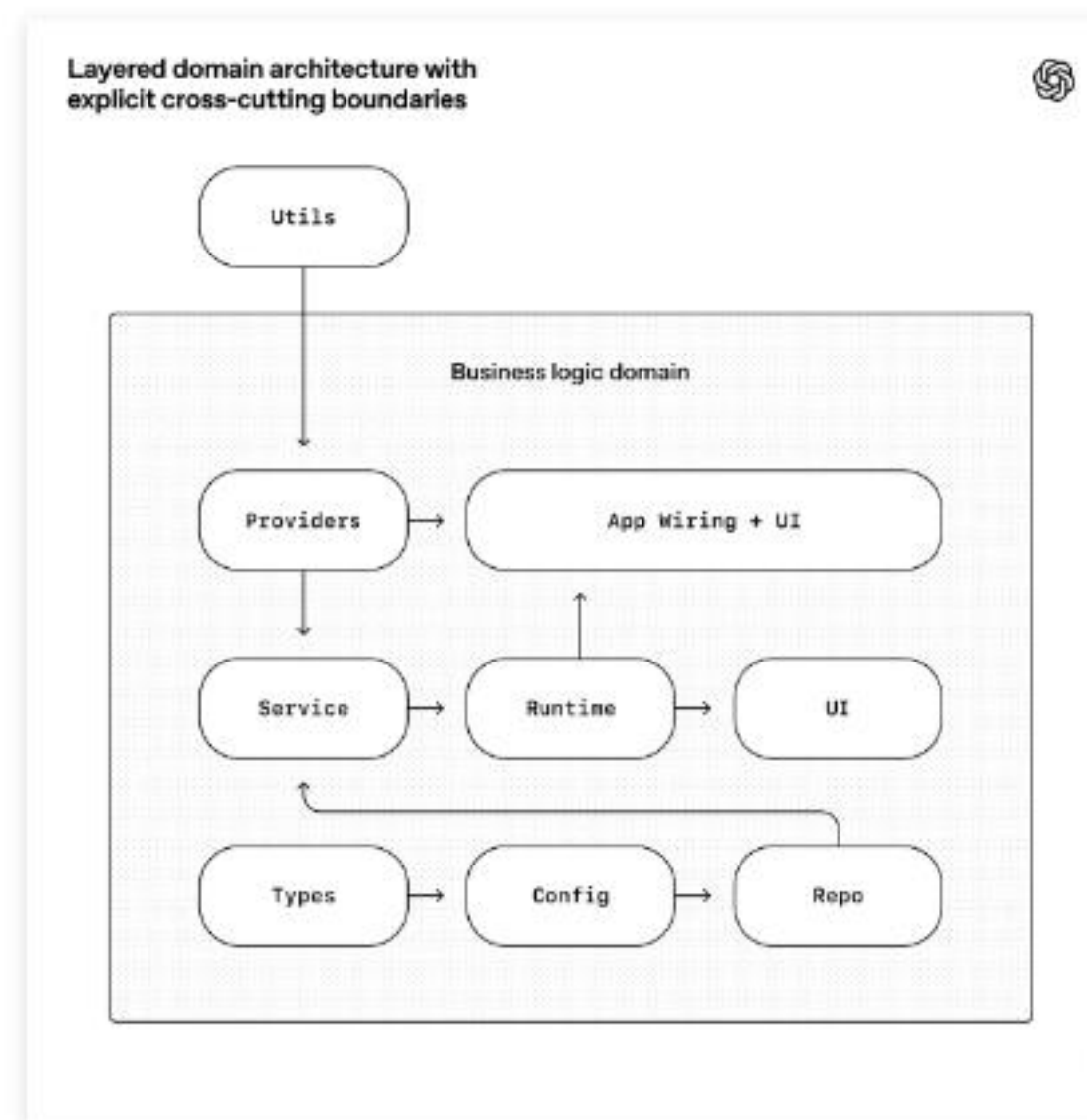
从单测全面升级到日志、指标、端到端行为及“品味”的综合评估。

🛡️ 约束

通过架构分层、依赖方向、命名规则的“机械化执行”，死守系统红线。

🧹 清理

对 AI 产生的冗余代码与系统垃圾进行持续回收，保持架构极致有序。



“如果你还在纠结怎么写 *prompt*，那你就是在高铁上纠结座椅颜色，却没人管刹车。”

Harness 工程：让系统更会纠错

Harness = Agent 的控制系统



Harness 工程不是教你把 Agent 训成“乖宝宝”，
而是 让整个系统更会纠错。

— AGENT 时代的软件工程控制论 —

三层进化链： 从“表达”到“控制”的范式转移

Prompt Engineering

— 第一层：单次调用的「表达工程」 —

🔗 核心关注点

极致把控单次模型调用的
输入与输出质量。

📁 工程化核心手段

01 任务描述

明确核心目标、设定绝对约束与角色边界。

02 推理引导

注入 Few-shot 示例，构建 Chain-of-Thought 逻辑路径。

03 格式工程

强制 JSON Schema 或 Markdown 表格输出约束。

— 范式转移：告别玄学咒语 —

传统 Prompt

工程化 Prompt

目标

得到一个答案



得到一个**可解析的结构**

手段

魔法短句 / 情绪价值



严格的边界定义与格式约束

局限

无法感知系统状态

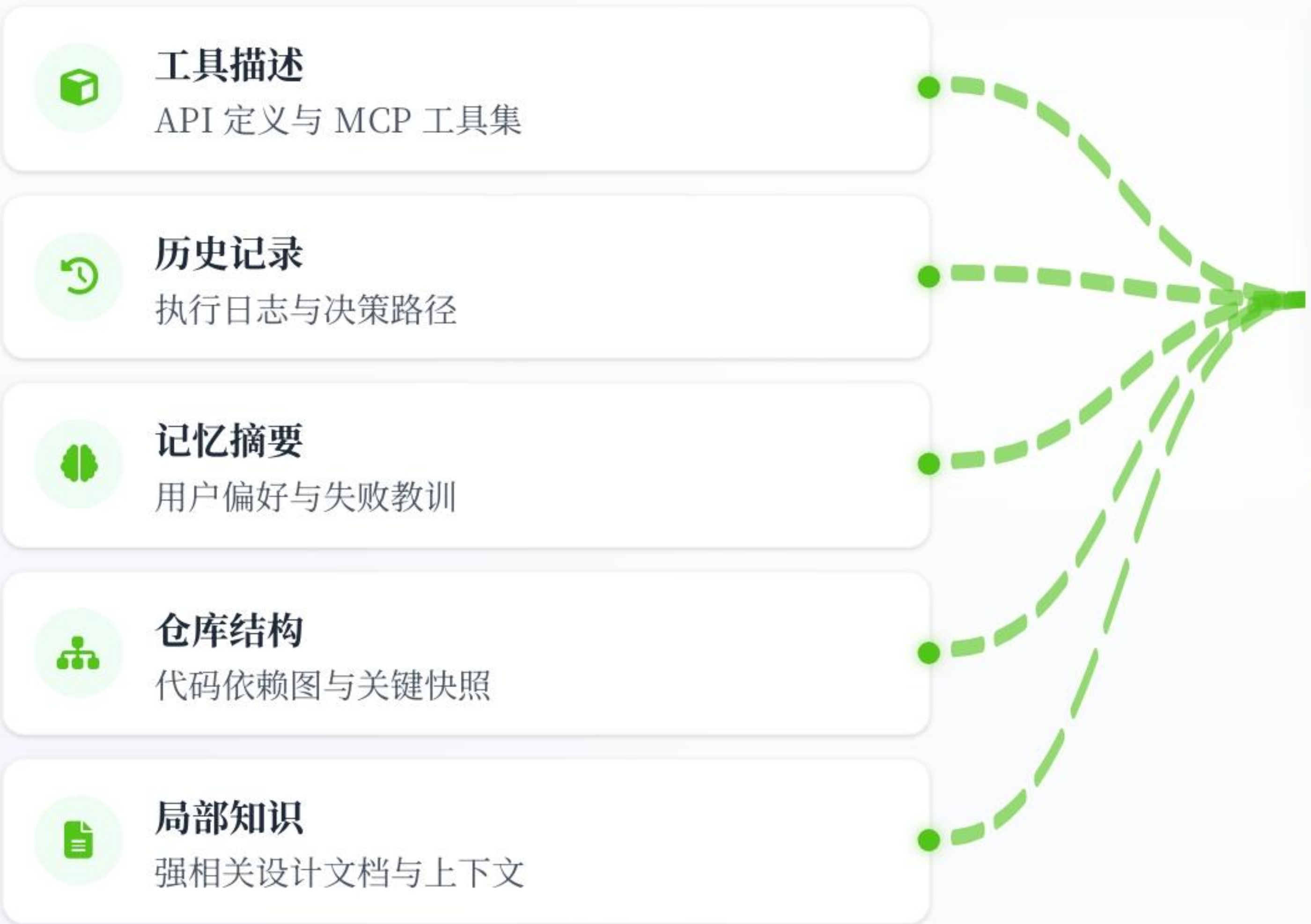


仅限单次交互，**无记忆性**

Context Engineering: 解决“喂什么”

第二层：决策点的“信息供给工程”

五大信息供给维度



Agent 执行瞬间

Agent 的质量取决于它在执行瞬间“看到了什么”。

高手不再纠结提示词，而是在纠结如何动态供给最相关的信息。

供给工程的两大挑战

渐进式披露

按需动态加载信息，避免一次性挤爆 Context 窗口。

污染隔离

自动识别并剔除陈旧、矛盾的规则与垃圾文档。

终局之战：Harness Engineering

第三层：系统级的“控制论工程”

核心本质

将“**纠错能力**”从人脑外化为系统闭环

“它不追求一次赌对，它追求的是**持续逼近正确**。”

🔧 Harness 控制回路逻辑



🛡️ 斩断三大工程顽疾



过早宣布胜利

代码写了但跑不通，缺乏真实环境的闭环验证



重复犯错

Agent 换班失忆，在同一个坑里反复跌倒



熵增腐烂

长任务积累系统熵，AI 垃圾随时间指数级堆积

维度打击：为什么 Harness 不是 Prompt 2.0

“Prompt 是沟通, Harness 是管理。一个是教员工怎么说话, 一个是给员工定 KPI、装监控、设审计流程”

对比维度	Prompt Engineering	Harness Engineering
管理对象	单次模型调用	整个 Agent 系统的演化
核心问题	这次怎么问更清楚?	系统偏了谁发现? 谁修?
时间尺度	毫秒级 / 秒级	小时级 / 天级 (长任务)
调优单位	提示词模板	反馈回路、规则、约束系统
成功标准	响应的正确性和	系统的鲁棒性和可扩展性



核心结论: Harness 是 系统级控制设计 , 而非“更长的系统提示”。

三层进化链总结：工程师的精力分配模型



01

初级 死磕 Prompt 魔法短句

将精力耗费在“怎么说”上，试图用一段完美的咒语解决所有复杂问题。

02

中级 构建动态 Context 检索系统

意识到模型能力的边界，开始关注“喂什么”，通过外部知识库弥补信息差。

03

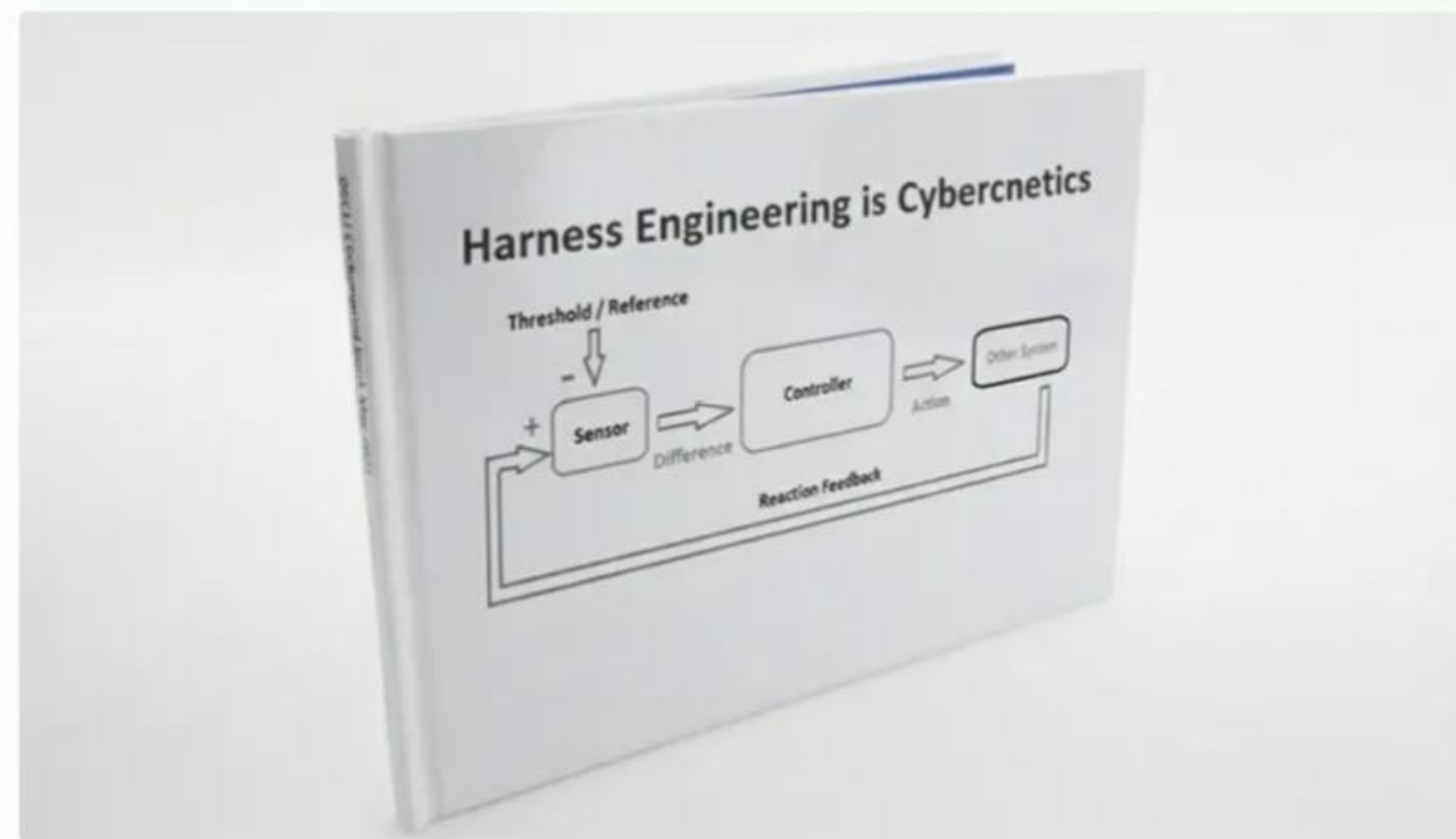
高级 设计自我纠错的 **Harness** 控制回路

跳出单次对话的局限，构建包含评估、反思与重试的完整工程闭环，解决“怎么赢”。

未来的顶级工程师，一定会把 **80%** 的精力，从“打磨表达”迁移到 **“设计闭环”** 上。

控制论主线： Harness Engineering 的灵魂回归

告别玄学调优，重塑 Agent 时代的
系统控制权。



控制论的起源：瓦特调速器

控制模式的代际跃迁：从「人肉执行器」到「系统设计师」

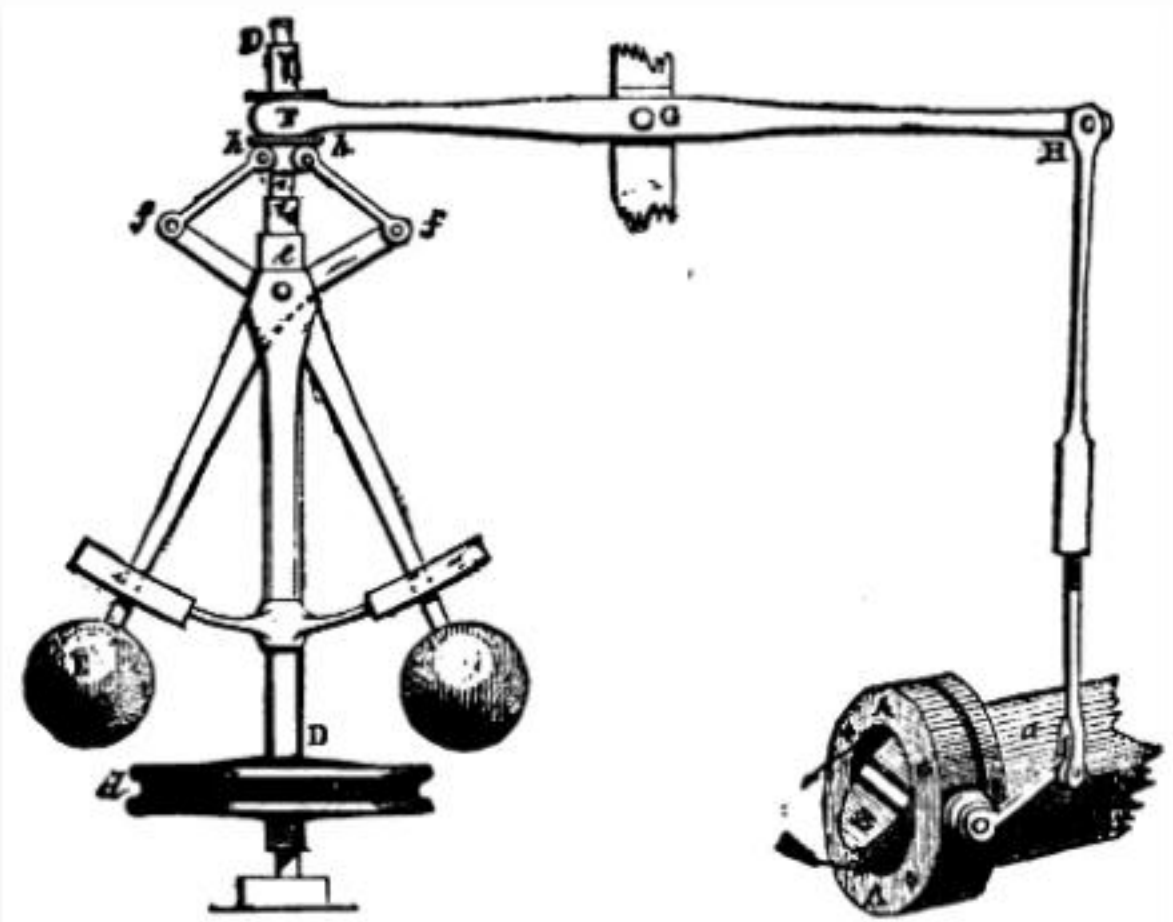


FIG. 4.—Governor and Throttle-Valve.

核心控制逻辑

目标 维持系统恒定转速

手段 将「输出偏差」直接转化为「输入修正」

复杂度上升到一定程度，人肉闭环必然失效。

维度	原始阶段 (手动控制)	进化阶段 (反馈控制)
核心角色	工人 = 实时执行器	工人 = 系统设计师
感知方式	人眼盯着表盘 易疲劳 / 易出错	飞球离心力 物理传感器
执行动作	手动旋转阀门 响应慢 / 滞后	机械杠杆自动调节 实时响应
反馈逻辑	发现偏航 → 人工干预	自动感知 → 自动修正

软件工程的控制论：Kubernetes Controller

声明期望状态，控制器持续监控真实状态并自动补齐、修复、回滚

I K8s 控制回路拆解

- **期望状态** (Desired State)
通过 YAML 声明目标
如：副本数 = 3
- **当前状态** (Current State)
传感器持续监控集群真实情况
- **调和动作** (Reconciliation)
对比偏差，自动执行
补齐、修复或回滚

I 工程师价值的「上移」

传统运维

命令式 (Imperative)

>_ ssh 登录

↻ 手动重启

📄 检查日志

关注「怎么做」(How)

现代运维 ✓

声明式 (Declarative)

📄 定义 SLA

</> 编写规格

🔄 监控反馈回路

关注「是什么」(What)



结论： Harness 是将“声明式控制”引入 Agent 运行环境。

Agent 时代的控制系统：Harness

工程师不再盯每个改动，而是定义完成、定义边界、定义偏航后的拉回机制。

壹

目标状态 定义什么叫“对”

通过 `feature_list.json` 等结构化文件，为 Agent 设定不可逾越的架构红线与最终目标。

贰

传感器 感知“哪里不对”

部署 CI、Lint、E2E 验证机制，实时监控 Agent 的代码产出，精准捕获偏航行为。

叁

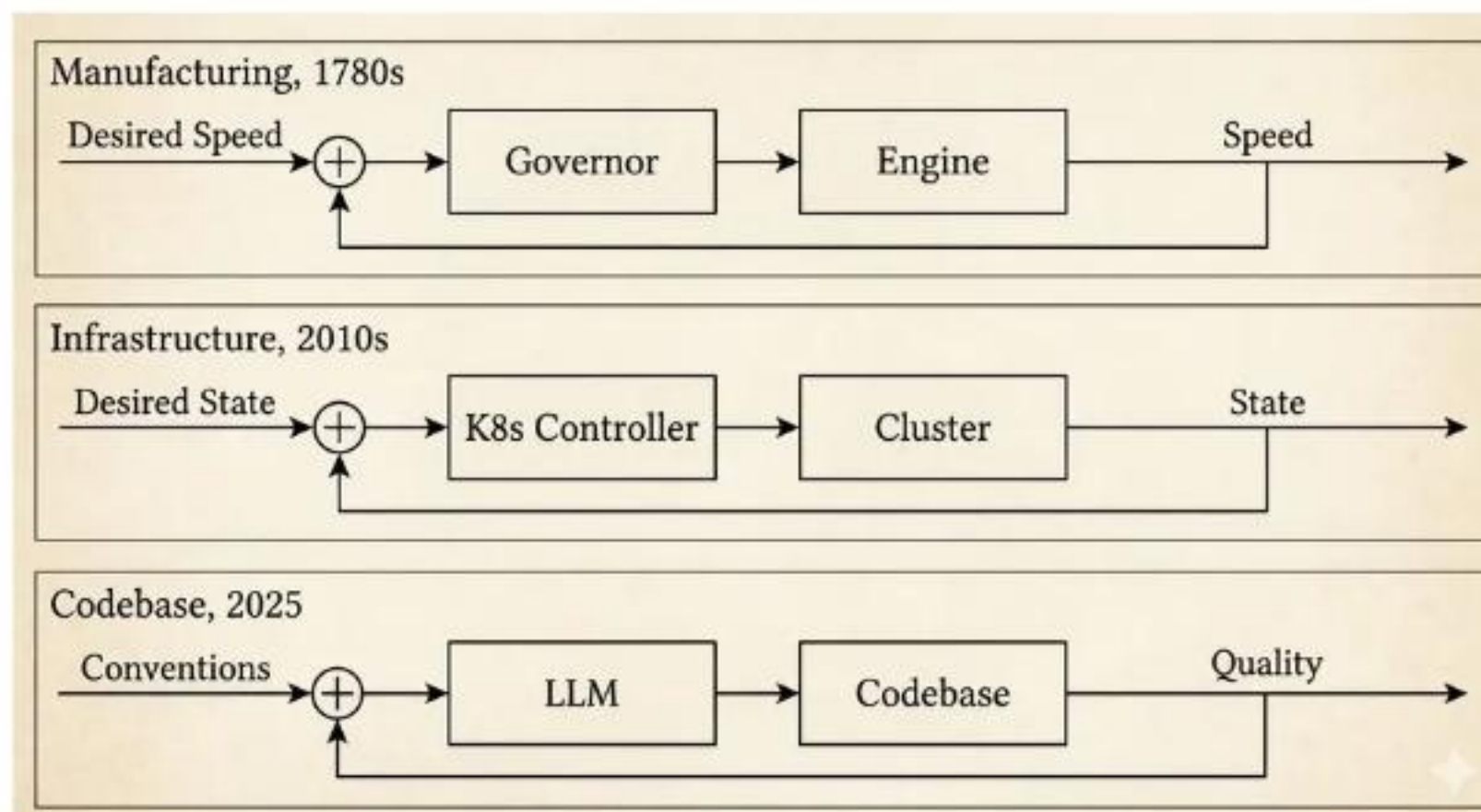
执行器 具备“修正能力”

当触发警报时，自动执行 Git Revert、重试逻辑或重构脚本，强行将 Agent 拉回正轨。

肆

反馈回路 持续逼近

感知 → 决策 → 执行 → 再验证



3.5 PR / 天

工程师的角色正在发生根本性蜕变：
从“代码编写者”转变为“反馈回路设计师”。

“人类掌舵，Agent 执行。”

系统复杂度的必然解法

技术演进的“上移”曲线

工业时代

蒸汽机转速失控

人工拧阀门



离心式调速器

云时代

微服务规模爆炸

手动重启 / 扩容



K8s 控制器

AI 时代 •

长任务 /
百万行代码

死磕 Prompt 调优



Agent
Harness



高杠杆策略

当 Agent 总是犯错，

别急着改 ~~Prompt~~

先问问自己：

控制系统搭好了吗？

系统复杂度越高，越需要「非人肉」的自动纠偏装置。

Harness: Agent 时代的软件工程终局

⚡ 核心公式

Coding Agent =
AI Model + **Harness**

最终拼的不是模型，而是你
定义“什么叫对”的能力。



工程师不是消失了，
而是上移到了“控制层”。

行动指南

01

外化判断

把脑子里的“品味”变成 Linter 规则。

02

结构化目标

把“需求”变成机器可读的 Feature List。

03

闭环反馈

让 Agent 在报错中自我进化，而不是靠人肉补丁。

04

Harness 的最小定义

把“正确”做成可持续运行的反馈系统

🔧 控制论核心回路

Harness = 目标状态 + 传感器 + 执行器 + 反馈回路

核心组件拆解

拆解 Agent 的“操作系统”

算力是引擎，
Harness 则是 方向盘 和 刹车。

模型是 CPU，算力再强，没有操作系统也跑不出生产力。这四块凑齐了，才叫工程化。

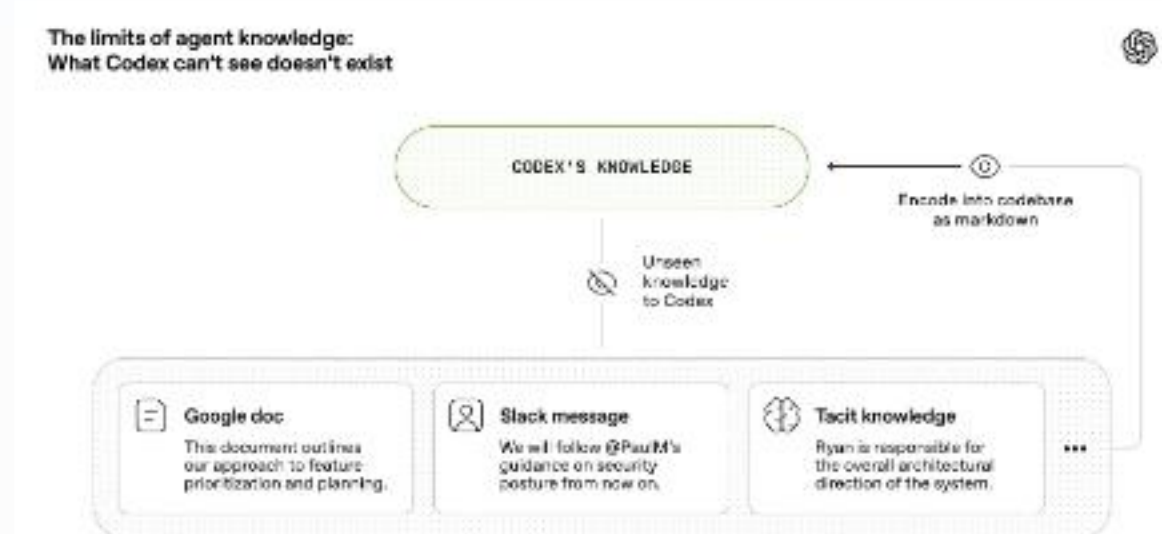


目标状态怎么写才算“可执行”

把“脑子里的感觉”变成“仓库里的标准”

- 1 完成 (Functional)** `feature_list.json`
明确界定哪些端到端行为必须成立，消除模糊地带。
- 2 可接受 (Quality)** `启动 < 800ms`
设定不可妥协的性能、稳定性与可观测性门槛。
- 3 符合架构 (Arch)** `ARCHITECTURE.md`
严格约束依赖方向、代码分层与模块边界。
- 4 改动干净 (Clean)** `QUALITY_SCORE.md`
确保不制造新的技术债务，且变更随时可安全回滚。

“别写‘请写得优雅’，那没用。
凡是没有写进仓库的规则，
对 Agent 来说一律不存在。”



Agent 知识的局限性：隐性知识必须编码到代码库中。

给 Agent 装上全天候监控堆栈

突破单元测试局限，构建四维传感器矩阵

静态防御

编译器与自定义 Linter，前置拦截架构违规。

动态验证

从单元到 E2E 自动化，全链路行为断言。

深度观测

Logs、Metrics 与 Traces，赋予 Agent SRE 级视野。

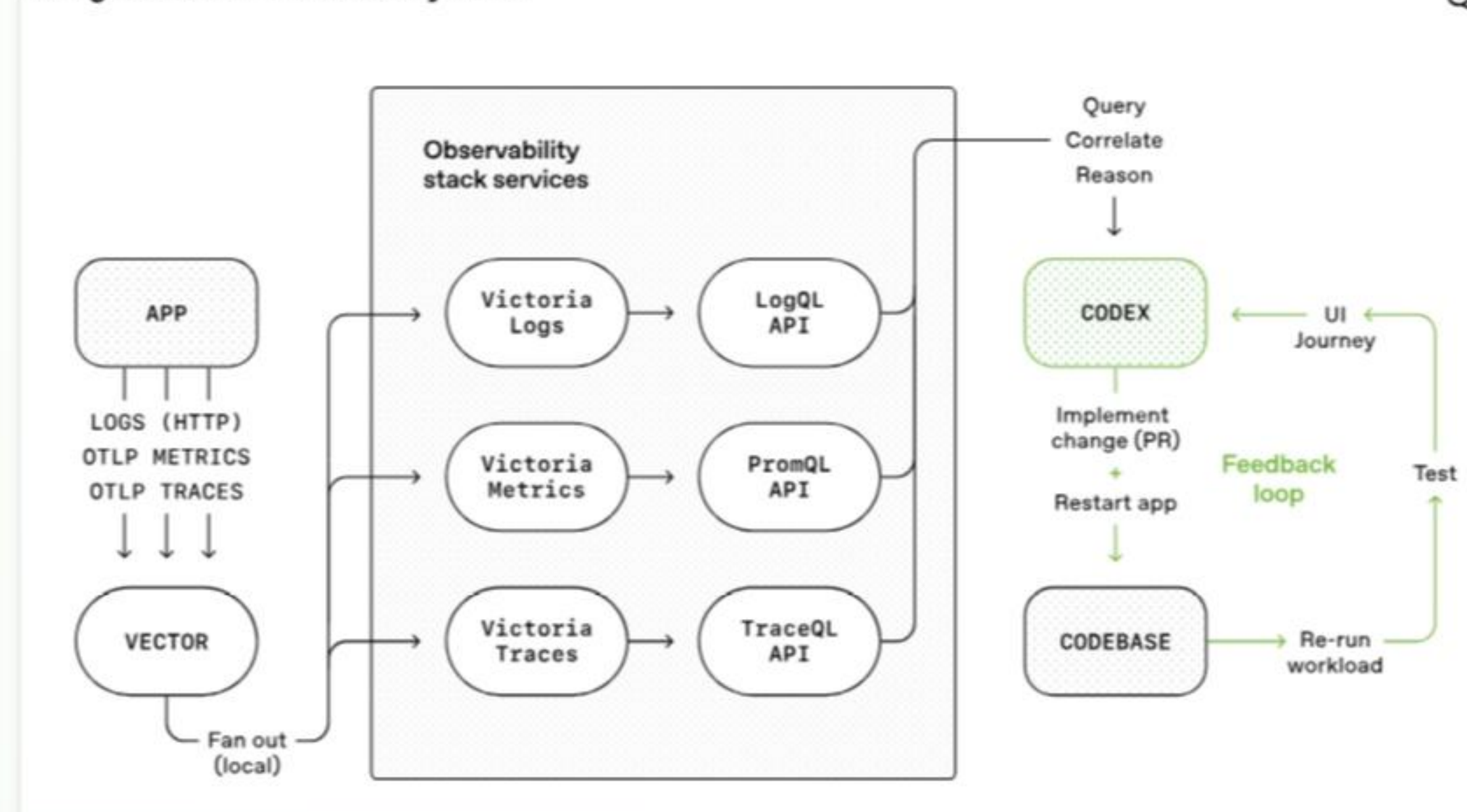
状态锚点

持久化决策路径，拒绝上下文迷失。

工程底线

所有传感器输出必须是 **强结构化信号**，彻底拒绝模糊的自然语言反馈。

Giving Codex a full observability stack



执行器动作库：Agent 的物理外挂

让 Agent 拥有“改变世界的手”



代码层

- 修改业务逻辑
- 补齐测试用例
- 重构坏代码



文档层

- 更新设计文档
- 记录决策历史
- 清理过时规则



环境层

- 调整 init.sh
- 修改 CI 配置
- 配置 Docker



流程层

- 开 PR / 响应 Review
- 自动合并代码
- Git Revert (回滚)



设计原则：所有动作必须 可审计、可回滚，优先封装脚本工具供 Agent 调用。

反馈回路：Agent 摆脱盲飞的唯一解

从“盲飞”到“自校准”的闭环

01 执行

Agent 基于上下文生成改动

02 感知

自动触发传感器 (测试/日志)

03 对比

观测结果 vs 目标状态

04 解析

将错误信号转化为可执行任务

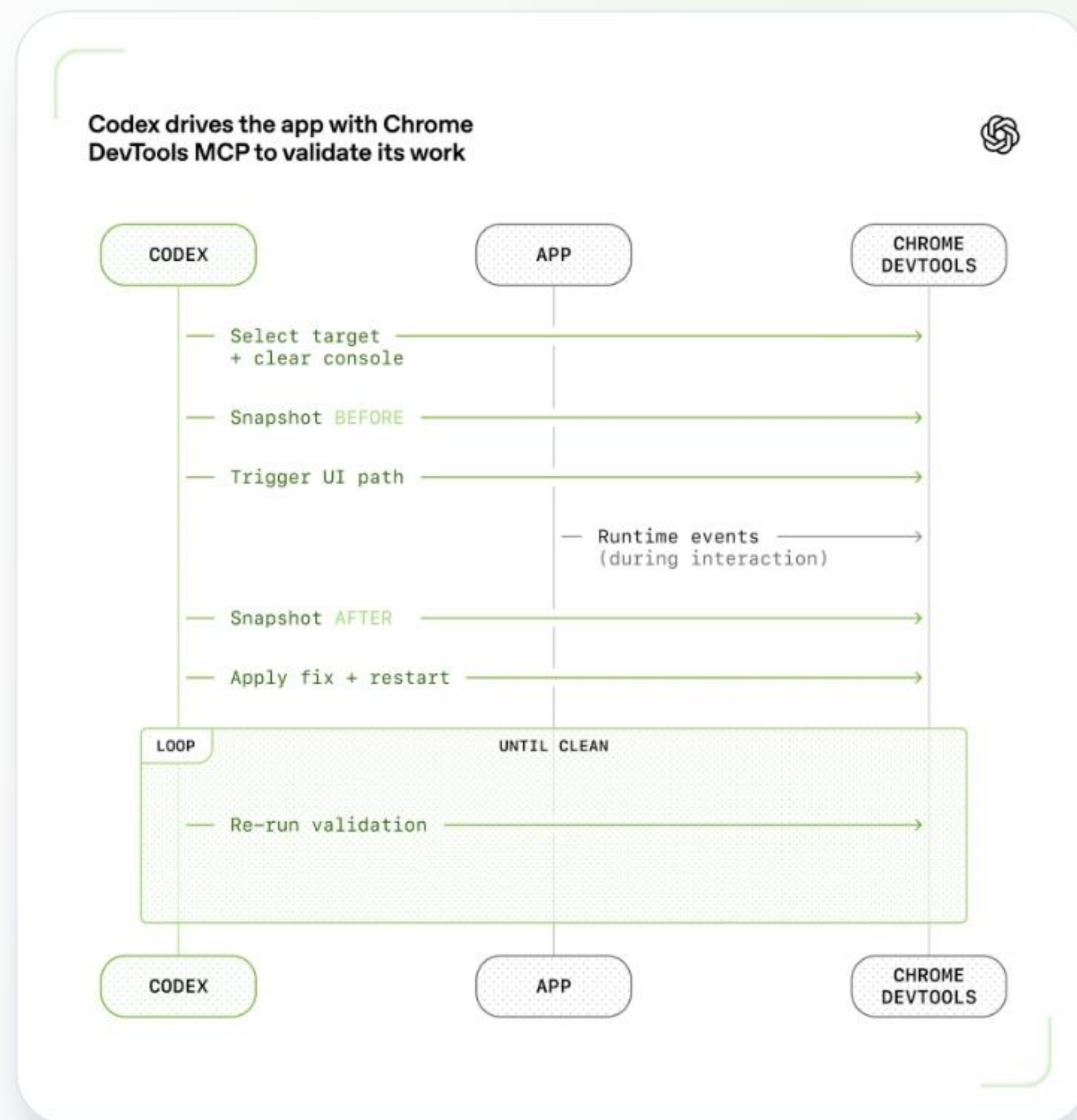
05 修正

调用执行器进行最小范围修复

06 再验证

重复循环直到系统状态纯净

“ 反馈回路是对抗熵增、实现长任务收敛的唯一手段。”



图：Codex 验证与修复闭环

范式转移：从“写代码”到“定规矩”

— 工程师的“上移”之路

↘ 传统能力 / 贬值中

↗ Harness 时代 / 升值中

手写复杂算法

→ • 定义“什么叫对”的标准

记忆框架 API

→ • 设计反馈回路与传感器

人肉 Code Review

→ • 编写架构约束与 Linter 规则

解决具体 Bug

→ • 治理系统熵增与垃圾回收

最终拼的不是模型，而是你定义 “正确” 的能力。

05

案例拆解： Agent 时代的 软件工程实战

"

*Humans steer.
Agents execute.*

人类掌舵，Agent 执行



传统码农模式

~~80% 时间手写代码 + 20% 调试~~



控制系统设计师模式

80% 设计环境与反馈 + 20% 审查

工程师不再拧阀门，而是设计那个能自动纠偏的调速器。

你不再是码农，而是控制系统设计师。

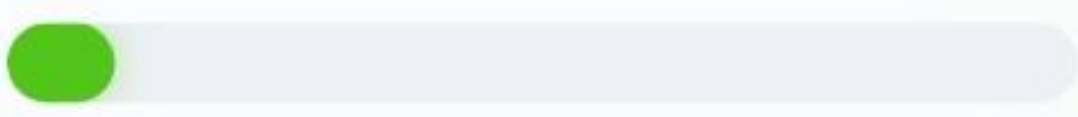
端到端高吞吐：百万行代码的疯狂实验

OpenAI 内部工程复盘

核心效能对比

交付周期 (从空仓库起步)

传统人力  约 50 个月

Agent  **5 个月**

人效比 (PRs / 工程师 / 天)

传统人力  0.5 PRs

Agent  **3.5 PRs**

</> 代码总量

100 万行

人类 0 行手写

成功合并 PR

1500+

打破"人多必乱"定律

Agent 端到端交付全景



业务逻辑 + 自动化测试

核心代码生成与全覆盖测试用例编写



CI/CD 配置 + 部署脚本

流水线搭建与自动化发布流程构建



动态文档生成

实时同步更新 ARCHITECTURE.md



内部可观测性仪表盘

系统监控、日志收集与运行状态可视化

知识外化与系统化

打造 Agent 的“系统记录仪”

“Agent 无法耳濡目染。凡是没有写进仓库的隐性知识，对它而言皆不存在。”

架构决策

脑内共识



ARCHITECTURE.md

产品 Sense

会议记录



PRODUCT_SENSE.md

质量标准

资深经验



QUALITY_SCORE.md

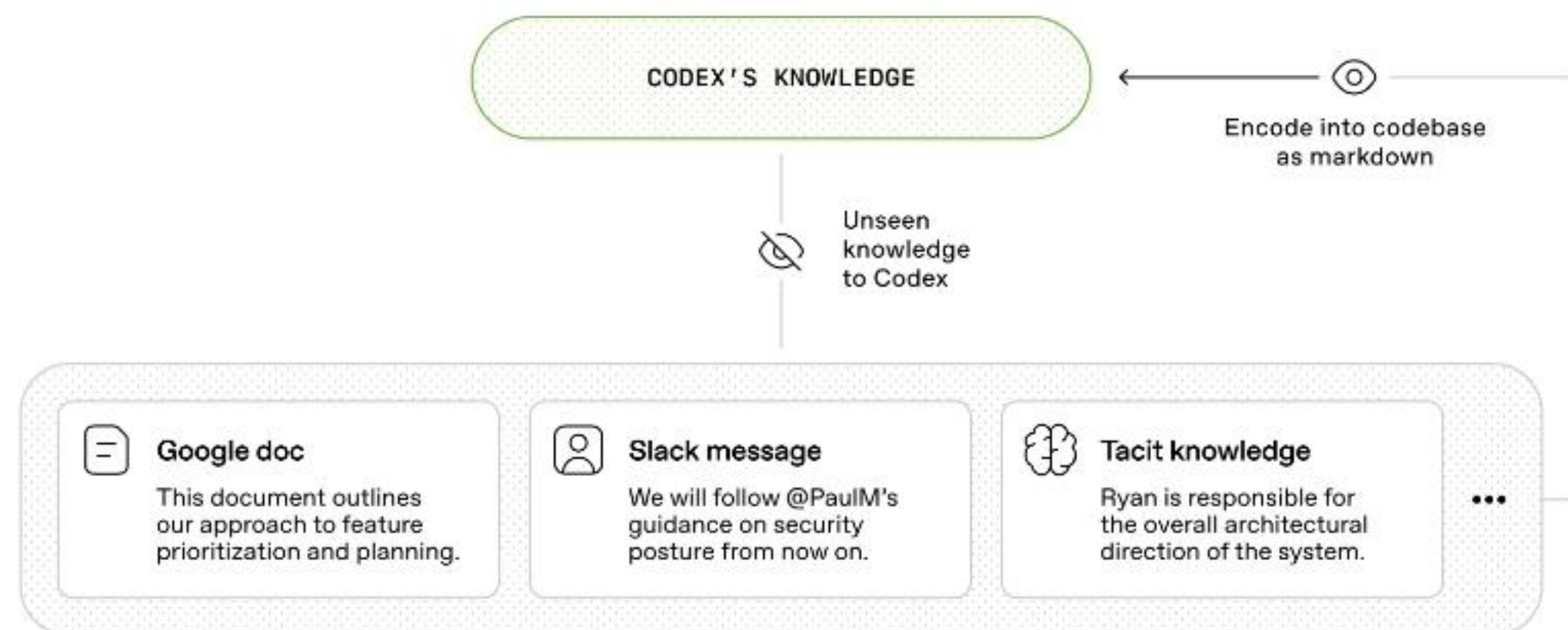
执行状态

进度同步



exec-plans/

The limits of agent knowledge:
What Codex can't see doesn't exist



渐进式披露：重塑 Agent 上下文预算

告别巨型手册，构建按需加载的知识树架构

⚠️ 传统反模式：巨型手册

试图把所有规范塞进单个巨型文件中，只会导致 **挤占上下文窗口**、**规则相互冲突**，并且随着项目迭代 **迅速腐烂**。模型最终会淹死在冗余的说明书里。

🌱 核心法则

让 Agent 像资深工程师一样 **『按需加载』** 知识，保持上下文绝对清爽。

AGENTS.md 全局入口

控制在 100 行以内，仅定义仓库的全局地图与核心行为原则。

docs/design-docs/ 📝 设计决策

存储系统的核心信念与架构选型，提供高维度的背景上下文。

docs/exec-plans/ 📋 执行日志

记录当前任务的进度、状态与决策过程，作为 Agent 的短期记忆区。

docs/references/ 📖 参考资料

沉淀 API 文档与第三方库指南，仅在需要调用时被精准检索介入。

告别“死文档”：Agent 时代的自动化知识治理

文档即代码：机械化维护

基础防线：自动化治理流水线



文档 Linter：强制格式规约

在提交阶段强制校验 Markdown 格式与元数据，将文档规范转化为**不可绕过的代码约束**。



链接检查器：阻断死链蔓延

CI 自动扫描全局交叉引用，确保知识网络无断点，**拒绝任何 404 误导 Agent**。



新鲜度监控：追踪生命周期

实时追踪最后更新时间，一旦触发阈值即刻提醒 Owner，**把文档卫生变成后台任务**。

高阶进化 · 主动清理

Doc-Gardening Agent

智能文档园丁

核心职责

持续对比代码实际变更与文档描述，消除两者间的“现实扭曲场”

执行动作

自动标记过时腐化的文档节点，并**主动提交重构或删除的 PR**。

“

只有 **‘活着’** 的文档，
对 Agent 才有价值。”

赋予 Agent “上帝视角”：全栈感知能力接入

应用可读性 (Application Legibility) 的系统级跃升



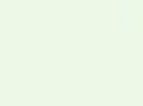
UI 交互层 Chrome DevTools

捕获 DOM 快照与实时截图，实现像素级状态感知。



日志分析层 LogQL

赋予 Agent 针对运行时错误的自主溯源与诊断能力



指标与链路层 PromQL / TraceQL

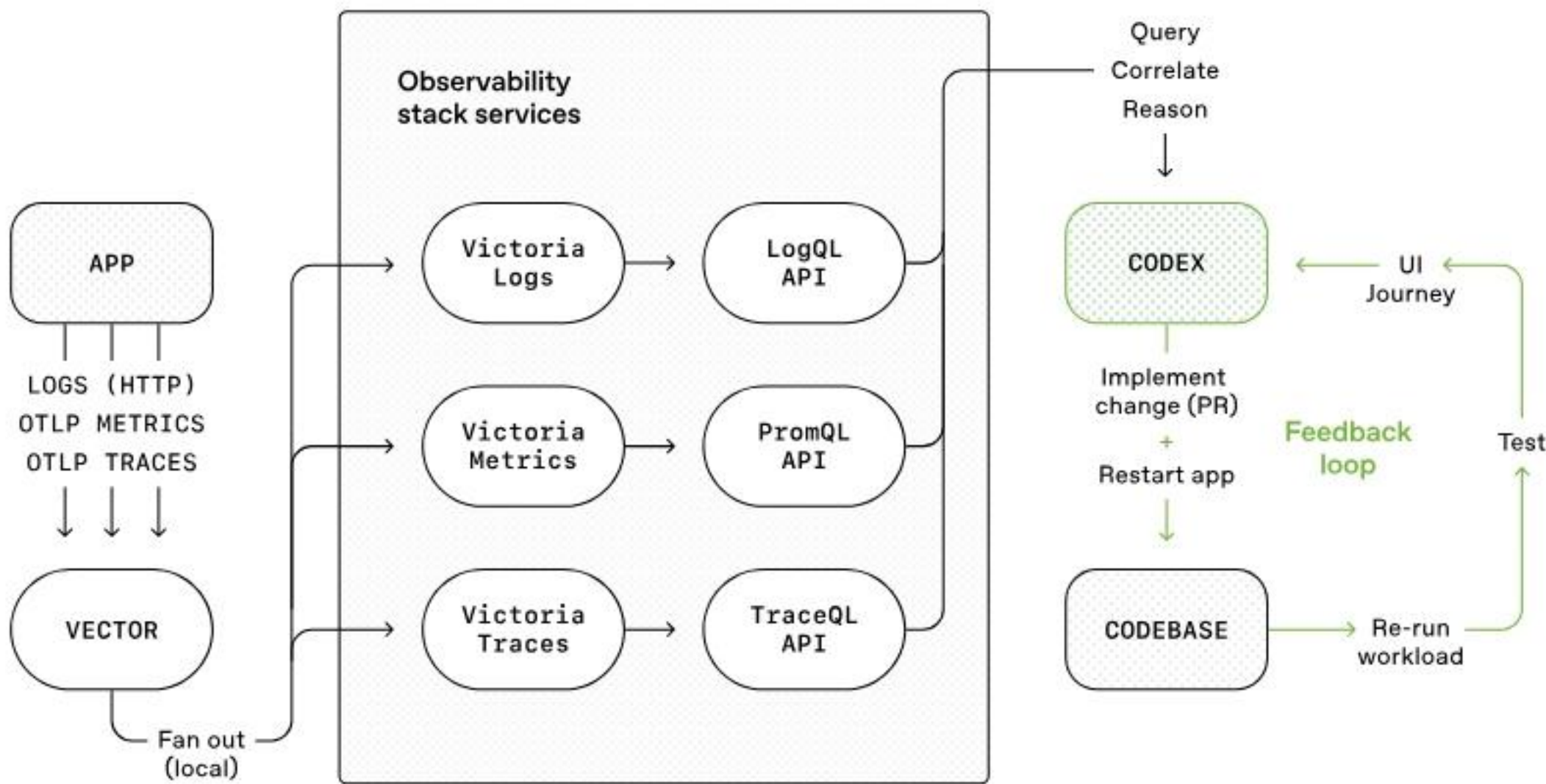
精准监控启动耗时 (**800ms**) 一键定位跨服务调用瓶颈



沙盒级环境隔离

为每个任务 (Worktree) 动态拉起独立的临时观测堆栈，确保 Agent 并发调试互不干扰。

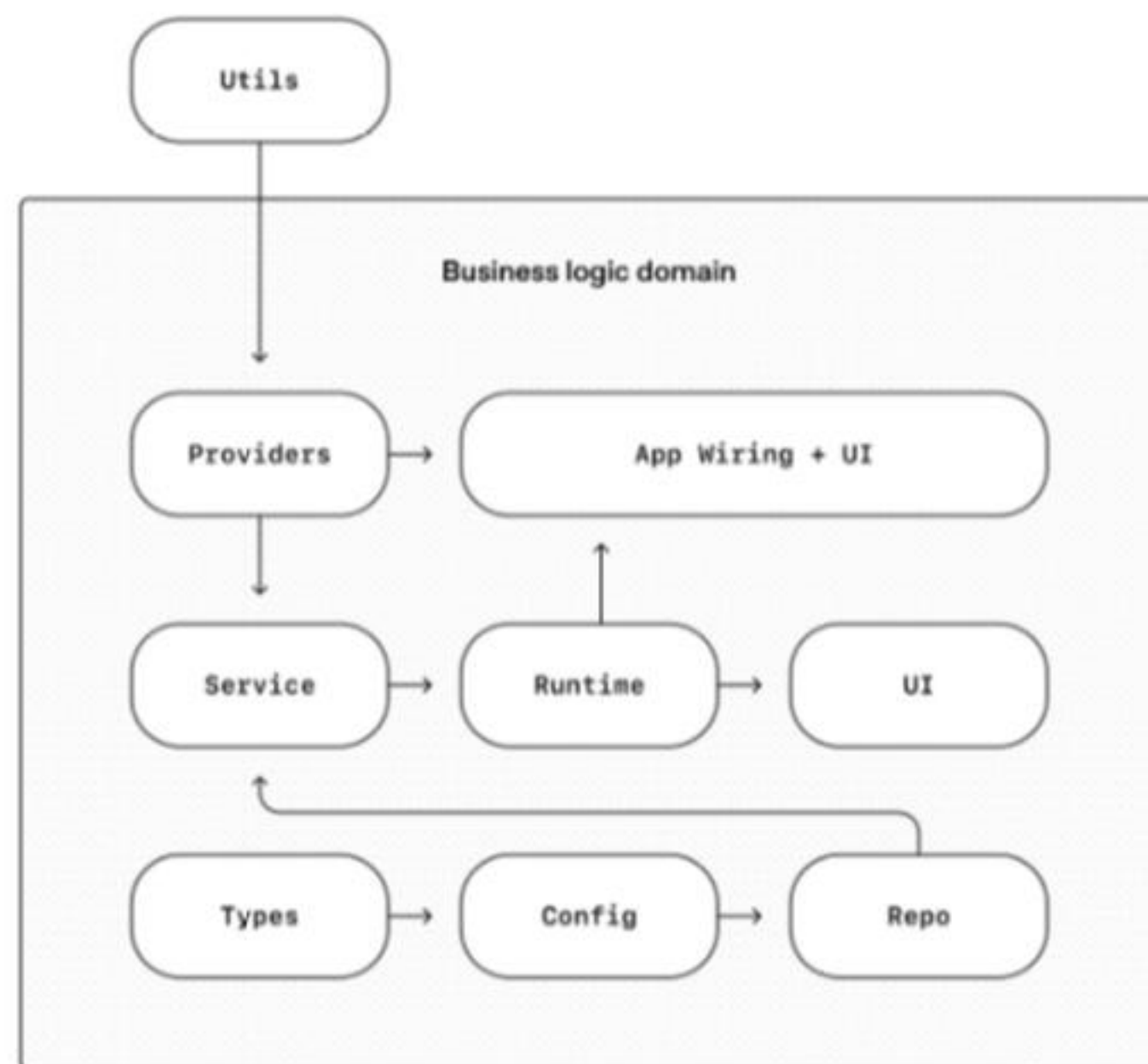
Giving Codex a full observability stack



系统架构拓扑图

Agent 时代的架构红线

Layered domain architecture with explicit cross-cutting boundaries



🔍 点击放大查看领域架构细节

📁 依赖方向的绝对红线

UI / Runtime

交互逻辑与环境



Service

业务逻辑编排



Repo / Config

持久化与配置



Types 底层

领域模型定义

🔧 品味不变量 (Taste Invariants)

✓ 别指望Agent自觉 必须强制执行结构化日志、文件大小限制与命名规范

📍 带路式报错：把老专家的经验写进编译器

Linters 拦截不仅要报错，更必须在信息中注入 **为什么错** 和 **如何修**

熵管理：AI 时代的垃圾回收 (GC)

—— 用“黄金原则”与后台清理任务持续纠偏，防止坏模式扩散

黄金原则 Golden Principles

01 Parse, don't validate

在边界处强制转换数据形状，拒绝隐式信任大模型的输出结构。

02 DRY (Don't Repeat Yourself)

严禁生成重复的 Helper 函数，强制复用共享工具链，避免逻辑碎片化。

03 Explicit over Implicit

所有约定必须显式编码，压缩 AI 的幻觉空间与猜测成本。

持续清理流程

扫描 Scan

后台 Agent 定期检索违背原则的代码模式。

重构 Refactor

自动提交小颗粒度 Refactor PR。

合并 Merge

人类进行秒级 Review 并自动合并。

长任务为什么总烂尾？

—— 长任务失败的结构性的根因

换班失忆

CONTEXT FRAGMENTATION

- ◆ 每个 Session 都是“纯净脑”，缺乏决策路径的连续性。
- ◆ 压缩后的上下文彻底丢失了关键的“为什么这么做”。

一次性幻觉

ONE-SHOT FALLACY

- ◆ 试图在单次调用中完成复杂功能，贪多嚼不烂。
- ◆ 导致：代码未测试、文档缺失、中间状态极其混乱。

● 致命后果 / FATAL CONSEQUENCE

下一轮 Agent 需要花费高达

80% 的 Token

去“猜历史”

这直接导致整个长任务的执行效率呈 指数级下降，最终走向崩溃。

拒绝废墟盖楼：Agent 时代的工程范式

两段式任务编排架构

01 先遣队 Initializer Agent

⚡ 核心动作

生成项目骨架、编写 `init.sh`、初始化进度文件。

📁 关键产出



02 主力部队 Coding Agent

🔄 执行逻辑

每次只领一个任务，严格遵循 实现 → 测试 → 提交 循环。

🛡️ 绝对约束



核心价值： 将长任务拆解为一系列可预测、可验证的增量步骤，让每一班 Agent 都在清爽的环境下接手。

拒绝 Agent 的「纸面胜利」

结构化完成标准 (Definition of Done)

● ● ● feature_list.json

```
{  
  
  "id": "feat-001",  
  "desc": "用户可创建新会话并收到回  
复",  
  "steps": ["点击新会话", "输入文本",  
"验证回复"],  
  "passes": false
```



剥夺篡改权限

Agent 仅拥有只读权限，严禁修改 `desc` 与 `steps` 字段，从根本上杜绝“偷偷改需求”的可能。



唯一状态变更权

将状态变更权死死绑定在测试链路上。仅在 端到端 (E2E) 测试 真实跑通后，才允许将状态翻转为 `true`。

这是硬指标：不准删需求，不准改描述。
彻底终结 Agent 伪造测试结果的幻觉。



Agent 换班不翻车：构建跨会话记忆桥梁

跨会话的“记忆桥梁”设计

核心工件组合

 **claude-progress.txt**

记录已完成、待办、关键决策与踩过的坑。

 **Git History**

提供版本化的代码演进轨迹与变更意图。

“新 Agent 上岗第一件事：读日志、看 Commit、跑脚本。三分钟对齐现状，拒绝盲猜，这就是工程化的连续性。”

标准启动流程 Bearings Check

01

```
$ pwd
```

执行命令以**确认环境**与当前工作目录。

02

```
$ cat claude-progress.txt
```

读取进度日志，**对齐进度**与历史上下文。

03

```
$ git log --oneline
```

运行 Git 命令，**扫描近期变动**与演进轨迹。

04

```
$ ./init.sh
```

运行初始化脚本，**验证环境健康度**。

init.sh: Agent 时代的“自愈”基建

🌱 环境自愈与心跳检查

环境坏了，代码写得再好也没用。

绝不在废墟上盖大楼。

—— 长任务稳定运行的底气



```
~ ./init.sh --check █
```

🔧 核心职责

- 自动安装与更新依赖库，抹平环境差异
- 静默启动本地开发服务器与数据库
- 运行基础 Sanity Test (冒烟测试)

🎮 工程博弈策略

🚫 失败优先处理

若脚本执行失败，Agent 必须**停止新功能开发**，强制转入环境修复模式

🔑 Token 护城河

避免 Agent 在每个 Session 中重复探索“如何运行项目”，极大节省上下文开销。

打破 Agent 的“纸面胜利”

终极传感器：端到端 (E2E) 闭环

集成 **Puppeteer / Playwright MCP** 工具，强制 Agent 站在**用户视角**评估交付质量，精准识别“代码逻辑正确但 UI 无法交互”的隐蔽 Bug。

01

环境唤醒

自动打开浏览器，建立交互基座

02

拟人交互

模拟真实用户点击，触发状态流转

03

视觉断言

捕获反馈，对比预期 UI 与真实 DOM

“没有 E2E 验证的 *Harness*, 充其量只是个半成品。”



Claude 通过 Puppeteer MCP 截取屏幕，验证克隆应用状态

终局推演

工程师角色变化

06

从「拧阀门」



进化到「掌舵」

工程师角色的「上移」路径

从直接执行者，转向系统设计者与规则制定者

“未来最贵的工程师，是能管理『会写代码的机器』的人。”



定 定义者

把“完成/干净”拆解为可验证的硬指标。

设 设计者

构建“目标-传感器-执行器”的闭环系统。

制 制定者

用机械化手段划定不可逾越的架构红线。

把隐性判断外化

让“品味”与“架构”可工程化

● 隐性判断外化矩阵

团队 Taste 从“写得顺眼” → **PRODUCT_SENSE.md**
与质量评分手册

架构偏好 从“默认分层” → **自定义 Linter**
与结构化依赖测试

干净标准 从“别留垃圾” → **Feature 清单**
与熵管理 Agent

决策背景 从“Slack 讨论” → **DESIGN.md**
与版本化执行计划

“凡是没有写进仓库的，
一律等于不存在。”

必须把隐性判断，全部“抽血”外化成机器能读懂的资产。

📌 行动指南

- 1 每次 Agent 犯错，就**加一条规则**。
- 2 报错信息必须包含“**为什么错**”和“**如何修**”。

降维打击：从代码防御到架构自治的闭环进化

软件工程的“控制论”回归



本质洞察

Harness 的出现，标志着软件工程终于在“高层语义”上，完成了控制论的终极闭环。过去只能靠人肉盯的架构与品味，现在首次实现了自动反馈与自我进化。

终局洞察

Harness 是生产力的最终乘数

$$\text{Coding Agent} = \text{AI Model} + \text{Harness}$$

同一模型, Harness 优化前

42%

成功率



同一模型, Harness 优化后

78%

成功率

Harness 的上限, 决定了你从模型里能榨出多少真实生产力。



Harness 工程 不是让 Agent 更聪明， 而是让系统更会纠错

“模型会迭代，框架会过时，
但只要系统里没有反馈回路，它迟早会失控。”

基于 OpenAI、Anthropic、Mitchell Hashimoto 实战总结

• 碳硅同传 | 期待与您在 AI 的春天再次相遇 •